

2026-06-10-SP4-cont-debluff-design

SP4-cont — De-Bluff Design: D-11 + D-12 (instance_manager.go execute* stubs)

Revision: 1 **Last modified:** 2026-06-10 **Maintainer:** SP4-cont READ-ONLY design subagent
Status: DESIGN ONLY — no code edited. Read-only investigation per task brief.

Target file: submodules/helix_agent/internal/clis/instance_manager.go (1452 lines) **Type constants:** submodules/helix_agent/internal/clis/types.go:19-69 **Proven SP4 helpers (reuse targets):** - extractPrompt(payload) — instance_manager.go:917-930 - resolveAgentBinary(typ, command) — instance_manager.go:936-958 (PATH lookup + HELIX_AGENT_BIN_<TYPE> fake-binary override) - runCLIAgent(inst, typ, command, args, payload) — instance_manager.go:964-1001 (real exec.CommandContext + combined output + exit code + honest error) **Proven RED-pin pattern (reuse target):** instance_manager_stub_pin_test.go (D-9 guards, GREEN-polarity + PIN_STUB_BLUFF=1 RED reproduction)

Governance: BLUFF-003 / CONST-035 / §11.4.99 (latest-source flags) / §11.4.115 (RED-on-broken-artifact polarity switch) / §11.4.135 (standing regression guard) / §11.4.123 (rock-solid-proof-or-research) / §11.4.6 (no-guessing — agents with no confirmed binary are flagged, never guessed).

0. Executive summary (answers the task's final-line questions)

- **D-12 stub method count: exactly 43.** Verified two independent ways: (a) 48 total execute* methods minus the 5 already-real (executeAider/executeClaudeCode/executeCodex/executeCline/executeOpenHands) = 43; (b) 43 "message": "<Agent> execution completed" literal lines (grep returns 44 total occurrences = 43 stub bodies + 1 doc-comment citation at instance_manager.go:913).
 - **NOTE on the task framing:** the brief says "qwencode" was among SP4's 5 fixed methods. That is NOT what the code shows — executeQwenCoder (instance_manager.go:1231) is STILL a stub returning "Qwen Coder execution completed". The 5 real methods are Aider/ClaudeCode/Codex/Cline/OpenHands (confirmed by instance_manager_stub_pin_test.go:74-78). Qwen Code IS in the research doc (qwen -p headless) so it is a high-confidence D-12 conversion, just not a done-already one.
- **Reuse vs bespoke: ALL 43 can reuse the existing runCLIAgent + resolveAgentBinary + extractPrompt helper trio unchanged** — the helper is fully generic over (CLIAgentType, command, args). NO new helper is required. The only per-method work is the 2-3 line body: prompt := extractPrompt(payload); return m.runCLIAgent(inst, Type<X>, "<cmd>", []string{<flags>}, payload). The split is therefore NOT helper-

vs-bespoke; it is **confirmed-binary (convert now) vs no-confirmed-binary (research/operator-block)**:

- **8 have a CONFIRMED CLI binary + non-interactive flags** in the 2026-06-10 research doc → convert now, identical to SP4.
- **35 have NO confirmed standalone CLI** (IDE extensions, hosted/web agents, raw model names, defunct projects) → see §3. They must NOT be guessed into an exec (§11.4.6); the honest options are (a) `exec.LookPath` honest-error stub via the same helper with a documented best-effort command name, or (b) explicit OPERATOR-BLOCKED / research-needed classification.
- **D-11 recommendation:** replace the "For now, allow all types" hard-coded enum `allowlist` (`instance_manager.go:387-398`) with a real availability check: `resolveAgentBinary(agentType, defaultCommandFor(agentType))` returning `err == nil` (i.e. real `exec.LookPath` per type, honoring the `HELIX_AGENT_BIN_<TYPE>` test override). Drives a single source of truth (the per-type command table) shared with the `execute*` methods. See §4.

1. Task 1 — Complete D-12 stub enumeration (43 methods, with line numbers)

Every method below currently returns the templated literal `{"status": "executed", "type": "<t>", "message": "<Agent> execution completed", "timestamp": ...}` with NO `exec.Dispatch` wired at `handleExecute` (`instance_manager.go:742-844`). Type constants at `types.go`.

#	Method	Def line	CLI AgentType (types.go)	Literature return
1	<code>executeKiro</code>	1033	<code>TypeKiro="kiro"</code> (24)	Kiro execution completed
2	<code>executeContinue</code>	1042	<code>TypeContinue="continue"</code> (25)	Continue execution completed
3	<code>executeSupermaven</code>	1051	<code>TypeSupermaven="supermaven"</code> (30)	Supermaven execution completed
4	<code>executeCursor</code>	1060	<code>TypeCursor="cursor"</code> (31)	Cursor execution completed
5	<code>executeWindsurf</code>	1069	<code>TypeWindsurf="windsurf"</code> (32)	Windsurf execution completed
6	<code>executeAugment</code>	1078	<code>TypeAugment="augment"</code> (33)	Augment execution completed
7	<code>executeSourcegraph</code>	1087	<code>TypeSourcegraph="sourcegraph"</code> (34)	Sourcegraph execution completed

#	Method	Def line	CLIAgentType (types.go)	Literature return
8	executeCodeium	1096	TypeCodeium="codeium" (35)	Codeium execu compl
9	executeTabnine	1105	TypeTabnine="tabnine" (36)	Tabnine execu compl
10	executeCodeGPT	1114	TypeCodeGPT="codegpt" (37)	CodeGPT execu compl
11	executeTwin	1123	TypeTwin="twin" (38)	Twin execu compl
12	executeDevin	1132	TypeDevin="devin" (39)	Devin execu compl
13	executeDevika	1141	TypeDevika="devika" (40)	Devika execu compl
14	executeSWEAgent	1150	TypeSWEAgent="swe_agent" (41)	SWEAgent execu compl
15	executeGPTPilot	1159	TypeGPTPilot="gpt_pilot" (42)	GPTPilot execu compl
16	executeMetamorph	1168	TypeMetamorph="metamorph" (43)	Metamorph execu compl
17	executeJunie	1177	TypeJunie="junie" (44)	Junie execu compl
18	executeAmazonQ	1186	TypeAmazonQ="amazon_q" (45)	AmazonQ execu compl
19	executeGitHubCopilot	1195	TypeGitHubCopilot="github_copilot" (46)	GitHubCopilot execu compl
20	executeJetBrainsAI	1204	TypeJetBrainsAI="jetbrains_ai" (47)	JetBrainsAI execu compl
21	executeCodeGemma	1213	TypeCodeGemma="codegemma" (48)	CodeGemma execu compl
22	executeStarCoder	1222	TypeStarCoder="starcoder" (49)	

#	Method	Def line	CLIAgentType (types.go)	Litera return
				StarC execu compl
23	executeQwenCoder	1231	TypeQwenCoder="qwencoder" (50)	Qwen execu compl
24	executeMistralCode	1240	TypeMistralCode="mistralcode" (51)	Mistra execu compl
25	executeGeminiAssist	1249	TypeGeminiAssist="gemini_assist" (52)	Gemin Assist execu compl
26	executeCodey	1258	TypeCodey="codey" (53)	Codey execu compl
27	executeLlamaCode	1267	TypeLlamaCode="llama_code" (54)	Llama execu compl
28	executeDeepSeekCoder	1276	TypeDeepSeekCoder="deepseek_coder" (55)	DeepS Coder execu compl
29	executeWizardCoder	1285	TypeWizardCoder="wizard_coder" (56)	Wizar execu compl
30	executePhind	1294	TypePhind="phind" (57)	Phind execu compl
31	executeCody	1303	TypeCody="cody" (58)	Cody execu compl
32	executeCursorSh	1312	TypeCursorSh="cursorsh" (59)	Cursor execu compl
33	executeTrae	1321	TypeTrae="trae" (60)	Trae execu compl
34	executeBlackbox	1330	TypeBlackbox="blackbox" (61)	Black execu compl
35	executeLovable	1339	TypeLovable="lovable" (62)	Lovab execu compl

#	Method	Def line	CLI AgentType (types.go)	Literature return
36	executeV0	1348	TypeV0="v0" (63)	V0 execution completed
37	executeTempo	1357	TypeTempo="tempo" (64)	Tempo execution completed
38	executeBolt	1366	TypeBolt="bolt" (65)	Bolt execution completed
39	executeReplitAgent	1375	TypeReplitAgent="replit_agent" (66)	Replit execution completed
40	executeIDX	1384	TypeIDX="idx" (67)	IDX execution completed
41	executeFirebaseStudio	1393	TypeFirebaseStudio="firebase_studio" (68)	Firebase Studio execution completed
42	executeCascade	1402	TypeCascade="cascade" (69)	Cascade execution completed
43	executeHelixAgent	1411	TypeHelixAgent="helixagent" (29)	Helix execution completed

Coverage cross-check (not stubs — out of D-12 scope): - TypeGoose (types.go:26), TypeForge (27), TypePlandex (28) HAVE type constants but **have NO execute* method and NO handleExecute case** → they fall to the default: honest error at instance_manager.go:842 ("execution not implemented for type"). These are not stub-bluffs (they return an honest error, not a fake success). Goose IS in the research doc (goose run -t) — adding it is *new feature work*, not de-bluffing, and is out of D-12 scope but worth a tracked follow-up.

2. Task 2/3 — Binary + flags per stub, helper-reuse grouping

GROUP A — CONFIRMED binary + non-interactive flags (8 methods) → convert NOW, identical to SP4

All cross-referenced to docs/research/2026-06-10-sdk-cli-currency.md PART B (fetched from official sources 2026-06-10). Each is a one-liner reusing runCLI Agent:

#	Method	command	args (non-interactive)	Research source (§11.4.99)
23	executeQwenCoder	qwen	[]string{"-p", prompt, "--"} prompt, "--"	research line 64 (qwen -p, --)

#	Method	command	args (non-interactive)	Research source (§11.4.99)
			output-format", "json"}	output-format json)
19	executeGitHubCopilot	copilot	[]string{"-p", prompt, "-s"} (no JSON format exists — -s = clean stdout)	research line 70 + 86 (honest negative: no JSON)
25	executeGeminiAssist	gemini	[]string{"-p", prompt, "--output-format", "json"}	research line 66 (gemini -p, --output-format json)

Note: only Qwen, Copilot, Gemini map cleanly from the research doc's 8 confirmed agents to a D-12 stub method (Claude/Codex/Cline/OpenHands already done; OpenCode/Crush/Goose have no matching stub method — OpenCode/Crush have no type constant at all, Goose is the default-case gap above). So **Group A is 3 methods, not 8**. Correcting the executive-summary estimate: deep cross-reference shows **only 3 stubs have a directly-confirmed binary+flags in the current research doc**. The remaining agents in §3 require a fresh §11.4.99 research pass before any exec command can be written.

§11.4.6 honesty correction: my initial “8 confirmed” was an over-count; after mapping each research-doc agent to an actual stub method, the confirmed set is 3 (Qwen, Copilot, Gemini-Assist). Codex/Claude/Cline/OpenHands are already-real; OpenCode/Crush/Goose have no D-12 stub to attach to. This is the rock-solid number.

GROUP B — PLAUSIBLE CLI binary, NOT yet in the research doc → research-needed (§11.4.99) before convert (≈9 methods)

These agents are known to ship *some* CLI but the exact current non-interactive invocation was NOT verified in the 2026-06-10 doc. They MUST get a fresh latest-source fetch (official docs) before an exec command is committed — guessing a flag is a §11.4.99 + §11.4.6 violation.

#	Method	Agent	Why research-needed
18	executeAmazonQ	Amazon Q Developer CLI (q)	q chat/q CLI exists; non-interactive prompt + output flags unverified
31	executeCody	Sourcegraph Cody (cody)	cody CLI exists historically; current headless form unverified
7	executeSourcegraph	Sourcegraph (src)	src CLI is code-search, not an agent runner — needs verification it even has an agent-exec mode
14	executeSWEAgent	SWE-agent	Python sweagent run CLI exists; exact current flags unverified

#	Method	Agent	Why research-needed
15	executeGTPilot	GPT-Pilot	Has a CLI endpoint; non-interactive form unverified
13	executeDevika	Devika	Open-source; server+UI primarily, CLI mode unverified
28	executeDeepSeekCoder	DeepSeek-Coder	A MODEL, not a CLI — only reachable via an ollama/API runner; see §3 ambiguity
27	executeLlamaCode	Code Llama	A MODEL — ollama run code llama is the realistic runner; needs design decision
21	executeCodeGemma / 22 executeStarCoder	models	Same model-not-CLI issue as Code Llama

GROUP C — NO standalone non-interactive CLI (IDE plugin / hosted / web / defunct) → honest-error or OPERATOR-BLOCKED (≈31 methods)

These have **no realistic headless CLI binary**. Converting them to a guessed exec is forbidden (§11.4.6). Two honest dispositions (operator decides per §11.4.101):

- **(C1) Honest-error via the same helper** — call `runCLIAgent` with a best-effort command name; on a host without that binary it returns the helper's honest "CLI not found on PATH" error (never fake success). This removes the bluff (no more fabricated "execution completed") WITHOUT claiming an exec form we cannot verify. The fake-binary test override still proves the exec wiring.
- **(C2) Explicit not-supported error** — return `fmt.Errorf("%s has no non-interactive CLI; use the <X> IDE/extension/API", typ)`. Honest, and clearer to users than a PATH-miss.

Recommendation: **C1** for everything (uniform with SP4, single code shape, testable with fake-binary injection, honest error when absent), and let the per-type command table (§4) carry an empty/sentinel command for the genuinely-CLI-less ones so C2's message is emitted instead.

Disposition	Methods
IDE extension only (no headless CLI)	<code>executeCursor</code> , <code>executeWindsurf</code> (Codeium's IDE), <code>executeCascade</code> (Windsurf's agent), <code>executeContinue</code> (Continue.dev — VS Code/JetBrains ext), <code>executeAugment</code> , <code>executeTabnine</code> , <code>executeCodeium</code> , <code>executeCodeGPT</code> , <code>executeJetBrainsAI</code> , <code>executeSupermaven</code> , <code>executeTwin</code> , <code>executeKiro</code> (AWS Kiro IDE), <code>executeTrae</code> (ByteDance IDE), <code>executeBlackbox</code> ,

Disposition	Methods
	executePhind, executeMetamorph, executeJunie (JetBrains agent)
Hosted / web-only agent (no local CLI)	executeDevin (Cognition hosted), executeLovable, executeV0 (Vercel web), executeTempo, executeBolt (StackBlitz web), executeReplitAgent (Replit web), executeIDX/ executeFirebaseStudio (Google web IDE), executeCursorSh
Raw model, no agent CLI	executeMistralCode, executeCodey (Google model), plus the §2 Group-B model rows if not promoted
In-repo agent (special-case)	executeHelixAgent — should exec the project's OWN helix/helixagent binary; command name MUST come from the project, NOT guessed. Treat as its own small task.

3. Task 3 — Fix design (reuse vs bespoke)

Finding: there is NO bespoke-helper work. One helper trio covers all 43. The SP4 helper `runCLIAgent(inst, typ, command, args, payload)` is fully generic; every method body is the same 2-line shape:

```
func (m *InstanceManager) executeX(inst *AgentInstance, payload
    interface{}) (interface{}, error) {
    prompt := extractPrompt(payload)
    return m.runCLIAgent(inst, TypeX, "<cmd>",
        []string{<flags...>}, payload) // §11.4.99 flags
}
```

The ONLY per-method variation is `(command, args)`. Therefore the real design question is **the command/flag table**, not the code structure. Two implementation options:

- **Option 1 (minimal, matches SP4 exactly):** edit each of the 43 bodies in place to the 2-line shape with its `(command, args)`. Pro: identical to the proven SP4 pattern + existing test harness. Con: 43 near-identical edits.
- **Option 2 (table-driven, recommended for D-11 synergy):** introduce one package-level table `var agentCLISpec = map[CLIAgentType]struct{ Command string; Args func(prompt string) []string }` and collapse the 43 methods + the `handleExecute` switch into a single generic dispatch + make `IsAgentTypeAvailable` (D-11) read the SAME table. Pro: single source of truth shared by D-11 and D-12, eliminates the 43-case switch and 43 method bodies, kills the

drift risk. Con: bigger diff, must preserve the `map[string]string` result shape the D-9 test asserts (`resultMessage` expects `map[string]string`).

Recommendation: Option 2 — it is the §11.4.111-style “single source of truth” win and makes D-11 fall out for free (§4). Keep `runCLIAgent`’s `map[string]string` return shape unchanged so `instance_manager_stub_pin_test.go:36-43` (`resultMessage`) keeps working.

Grouping for execution batches (per §11.4.103 parallel streams): - **Batch 1 (3 methods, convert now):** Qwen, GitHub Copilot, Gemini-Assist — confirmed flags, copy SP4. - **Batch 2 (≈9, research-then-convert):** Group B — one §11.4.99 fetch per agent, then convert. Model-vs-CLI rows (Code Llama / StarCoder / CodeGemma / DeepSeek-Coder) need an operator decision: run via `ollama run <model>?` If yes, they become a confirmed `ollama` exec; if no, they go to Group C. - **Batch 3 (≈31, de-bluff to honest-error):** Group C — C1 uniform shape; emits honest “CLI not found / not supported” instead of fabricated success. This REMOVES the bluff even though no real agent runs, which is the constitutionally-correct state (honest error > fake success, BLUFF-003). - **Batch 4 (1, special):** `executeHelixAgent` — exec the project’s own binary; command name supplied by config, not guessed.

4. Task 4 — D-11 design (`IsAgentTypeAvailable`, `instance_manager.go:387-398`)

Current bluff:

```
func (m *InstanceManager) IsAgentTypeAvailable(agentType
    AgentType) bool {
    // Check if there's a pool for this type
    // For now, allow all types defined in the enum
    switch agentType {
    case TypeAider, TypeClaudeCode, TypeCodex, TypeCline,
        TypeOpenHands, TypeKiro, TypeContinue, TypeHelixAgent:
        return true
    default:
        return false
    }
}
```

Two defects: (1) the “For now, allow all types” literal is a CONST-035 anti-pattern marker; (2) it returns `true` for `TypeKiro`/`TypeContinue`/`TypeHelixAgent` whose `execute` methods are stubs — i.e. it claims availability for agents that cannot actually run. It is a hard-coded allowlist, not a real check.

Recommendation — real `exec.LookPath`-based availability via the shared command table:

```
func (m *InstanceManager) IsAgentTypeAvailable(agentType
    CLIAgentType) bool {
    spec, ok := agentCLISpec[agentType] // the §3 Option-2 table
    if !ok || spec.Command == "" {
        return false // no CLI mapping → genuinely unavailable
    }
}
```

```

    }
    _, err := resolveAgentBinary(agentType,
        spec.Command) // real PATH lookup +
        HELIX_AGENT_BIN_<TYPE> override
    return err == nil
}

```

Why this is the right shape: - **Real check, not an allowlist** — answers the actual question (“can this agent run on THIS host right now?”) via `exec.LookPath` (already inside `resolveAgentBinary`, instance `_manager.go:953`). - **Honors the test override** — `HELIX_AGENT_BIN_<TYPE>` lets unit tests inject a fake binary, so D-11 gets the same anti-bluff fake-binary test treatment as D-9/D-12. - **Single source of truth** — shares the §3 table with the `execute*` dispatch; an agent is “available” iff it has a real, resolvable binary, exactly matching what `runCLIAgent` will attempt. No drift. - **AgentType vs CLIAgentType note:** the current signature uses `AgentType`; confirm whether `AgentType` and `CLIAgentType` are the same alias or need a conversion — flag for the implementer (read `types.go` aliasing before edit). If they differ, the table key type must match.

Caveat (§11.4.6): `exec.LookPath` proves the binary is *on PATH*, not that it is *authenticated/working*. That is the honest, bounded claim “availability” should make — a deeper health probe (e.g. `--version`) is a separate, optional enhancement, not required to kill the bluff.

5. Task 5 — RED-first plan per §11.4.115 (batched pin guards)

Reuse the EXISTING `instance_manager_stub_pin_test.go` harness verbatim — `writeFakeAgentBin` (line 47), `resultMessage` (36), the GREEN guard shape (62-115), the absent-binary honest-error guard (119-146), and the `PIN_STUB_BLUFF=1` RED reproduction (152-191). The work is to EXTEND the three `cases` slices and the `bluffLiterals` slice to cover the converted methods.

RED → GREEN polarity per §11.4.115 (one source, two roles): - **RED (pre-fix, `PIN_STUB_BLUFF=1`):** assert each converted method STILL returns its “<Agent> execution completed” literal on the current stub artifact → FAILs (defect genuinely present). This is the proof the guard is real, not synthetic. - **GREEN (post-fix, default):** inject `HELIX_AGENT_BIN_<TYPE>` fake binary echoing an unforgeable marker; assert (a) the marker IS in the result, (b) the literal is NEVER returned, (c) the prompt was forwarded. The bug-catcher IS the standing regression guard (§11.4.135).

Batched test additions (one t.Run per method, table-extended): - **Batch 1 (Group A, 3):** add `{"qwencoder", TypeQwenCoder, "HELIX_AGENT_BIN_QWENCODER", mgr.executeQwenCoder}` etc. to BOTH the `exec-real` and `absent-binary` case tables, and the literals to `bluffLiterals`. Full GREEN guard (marker present) applies — these really `exec.` - **Batch 3 (Group C, honest-error class):** for the C1 honest-error methods, the GREEN assertion is the **absent-binary honest-error** path (`TestD9...AbsentBinaryIsHonestError` shape) PLUS, when a fake binary IS injected, the marker-present path. The key anti-bluff assertion for C1 is: with NO binary, an ERROR is returned (never the literal, never fake success). Add these to the absent-binary table. - **Per §11.4.135:** every converted method gets a permanent row in the standing guard in the SAME commit as its conversion. A conversion landing without its guard row is a §11.4.123 violation. - **Paired §1.1 mutation:** revert any one converted body to its literal

→ the corresponding `t`. Run MUST FAIL (marker absent / literal returned). This is already how the D-9 test is structured; extending the tables inherits the mutation property for free.

Build/vet/test commands (from submodules/helix_agent/): `go vet ./internal/clis/...` → `go test -count=1 ./internal/clis/...` (GREEN guards) → `PIN_STUB_BLUFF=1 go test -run StubsAreBluffs -count=1 ./internal/clis/...` on the pre-fix artifact (RED proof).

6. Task 6 — Sequence, effort, operator/research-blocked flags

Sequence (priority-ordered per §11.4.42 / §11.4.132 — highest-confidence + most-visible first): 1. **D-11 + §3 command table** (foundation; D-12 Option 2 depends on it). ~2-3h incl. AgentType/CLIAgentType reconciliation + fake-binary D-11 test. 2. **Batch 1 — Group A 3 methods** (Qwen, Copilot, Gemini-Assist): convert + extend pin guards. ~1h (copy SP4). 3. **Batch 3 — Group C ~31 methods** to C1 honest-error: mechanical, table-driven, one absent-binary guard pass. ~2-3h. 4. **Batch 4 — executeHelixAgent**: needs the project's own binary name (config-sourced). ~1h once command confirmed. 5. **Batch 2 — Group B ~9 methods**: BLOCKED on a fresh §11.4.99 research pass per agent + operator decision on the model-vs-CLI rows. ~1h research + ~1h convert PER confirmed agent; model rows may collapse into Group C if operator declines an ollama runner.

Total effort estimate: ~9-12h for D-11 + Batches 1/3/4 (the de-bluffable-now set, 35 methods). Batch 2 (~9 methods) is gated on research + 1 operator decision and is additive.

OPERATOR / RESEARCH-BLOCKED flags (§11.4.6 / §11.4.99 / §11.4.101): - **NO confirmed binary/flags — research-needed before any exec** (Group B): `executeAmazonQ`, `executeCody`, `executeSourcegraph`, `executeSWEAgent`, `executeGPTPilot`, `executeDevika`. - **MODEL-not-CLI — operator decision required** (run via ollama `run <model>` or treat as no-CLI): `executeLlamaCode` (codellama), `executeStarCoder`, `executeCodeGemma`, `executeDeepSeekCoder`, `executeMistralCode`, `executeCodey`. - **NO standalone headless CLI — de-bluff to honest-error, NOT a real exec** (Group C, 31): all IDE-extension / hosted-web rows in §2 Group C. These are NOT “fixable into a working exec”; the constitutionally-correct fix is the honest error. Do NOT let a future cycle mis-classify them as “broken” (§11.4.112 honest-boundary): they are structurally CLI-less, not buggy. - `executeHelixAgent` — command name is project-owned; supply from config, never guess (§11.4.6).

7. FINAL ANSWER (task's required closing line)

- **D-12 method count: exactly 43** stub `execute*` methods (Kiro...HelixAgent), `instance_manager.go`: 1033-1417. (48 total `execute*` – 5 already-real.)
- **Helper reuse vs bespoke: ALL 43 reuse the existing `runCLIAgent/resolveAgentBinary/extractPrompt trio` — ZERO bespoke helper work.** Each body is the same 2-line SP4 shape; only (`command`, `args`) varies. The real split is by binary-confidence, NOT by helper:
 - **3 convert-now** (confirmed binary+flags in the 2026-06-10 research doc):
`executeQwenCoder(qwen -p --output-format json)`,
`executeGitHubCopilot(copilot -p <prompt> -s)`,
`executeGeminiAssist(gemini -p --output-format json)`.

- **~9 research-needed** (§11.4.99 fetch first; incl. 6 model-vs-CLI rows needing an operator o l l a m a - o r - n o t decision): AmazonQ, Cody, Sourcegraph, SWEAgent, GPTPilot, Devika + LlamaCode/StarCoder/CodeGemma/DeepSeekCoder/MistralCode/Codey.
- **~31 de-bluff-to-honest-error** (structurally no headless CLI — IDE extensions / hosted-web): convert to the helper's honest "CLI not found / not supported" error, which REMOVES the bluff without faking an exec.
- **1 special** (executeHelixAgent) — exec the project's own binary, config-sourced name.
- **D-11 recommendation:** replace the "For now, allow all types" enum allowlist (instance_manager.go:387-398) with a REAL per-type exec.LookPath check driven by the shared §3 command table: IsAgentTypeAvailable(t) = resolveAgentBinary(t, agentCLISpec[t].Command) == nil (empty/absent command ⇒ false). This honors the HELIX_AGENT_BIN_<TYPE> test override, is anti-bluff-testable with fake-binary injection, and shares one source of truth with the D-12 dispatch. Confirm the AgentType vs CLI AgentType signature type before editing.

Sources verified 2026-06-10

- docs/research/2026-06-10-sdk-cli-currency.md (PART B — Qwen/Copilot/Gemini/Codex/Claude/OpenCode/Crush/Goose non-interactive flags, fetched from official docs 2026-06-10).
- Code: submodules/helix_agent/internal/clis/instance_manager.go (lines cited inline), .../types.go:19-69, .../instance_manager_stub_pin_test.go.
- Negative finding (§11.4.6): no current official non-interactive CLI flags were verified on 2026-06-10 for Group B/C agents — those rows are flagged research-needed/CLI-less, NOT guessed.